From Your Laptop to the Cloud

Local Spark · Amazon EMR · SageMaker

PPOL 5206 · Massive Data Fundamentals · Week 12

April 9, 2026

The Three-Layer Map

1 Local Spark

You Manage: Java, Spark, env vars, ports
Managed For You: Nothing
When: Learning, debugging, offline dev

2 EMR on EC2

You Manage: Cluster size, instance types, IAM
Managed For You: OS patching, Spark install, HDFS
When: Production, full control

3 EMR Serverless

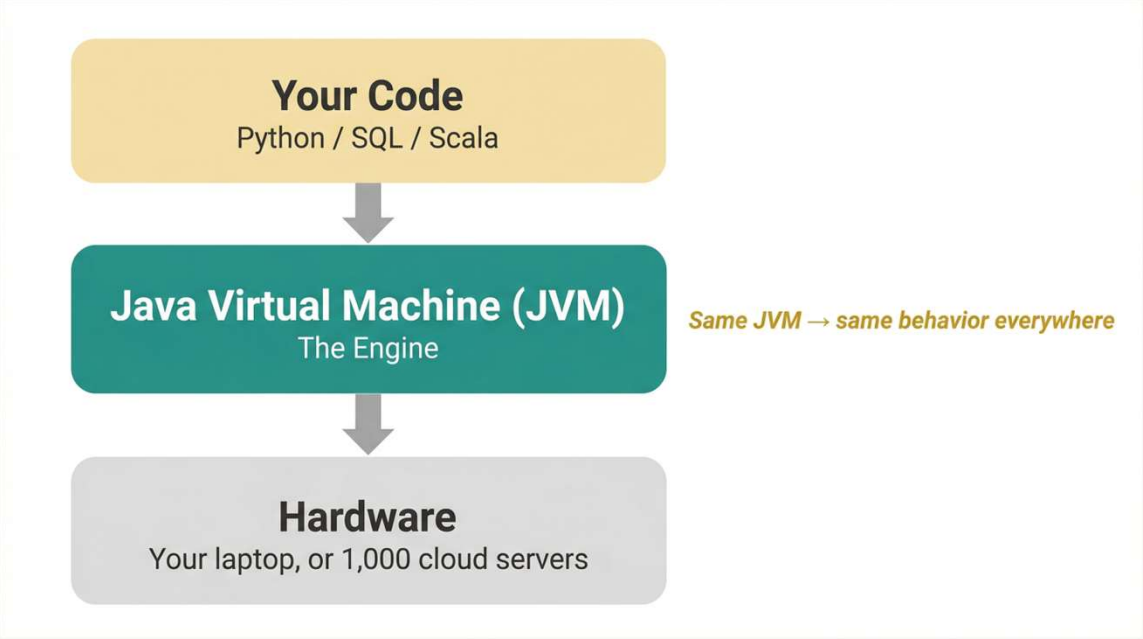
You Manage: Your Spark code
Managed For You: Everything else
When: Production, cost + simplicity

Same Spark code works at all three layers. Only the connection changes.

Part 1: Your Laptop as a Server

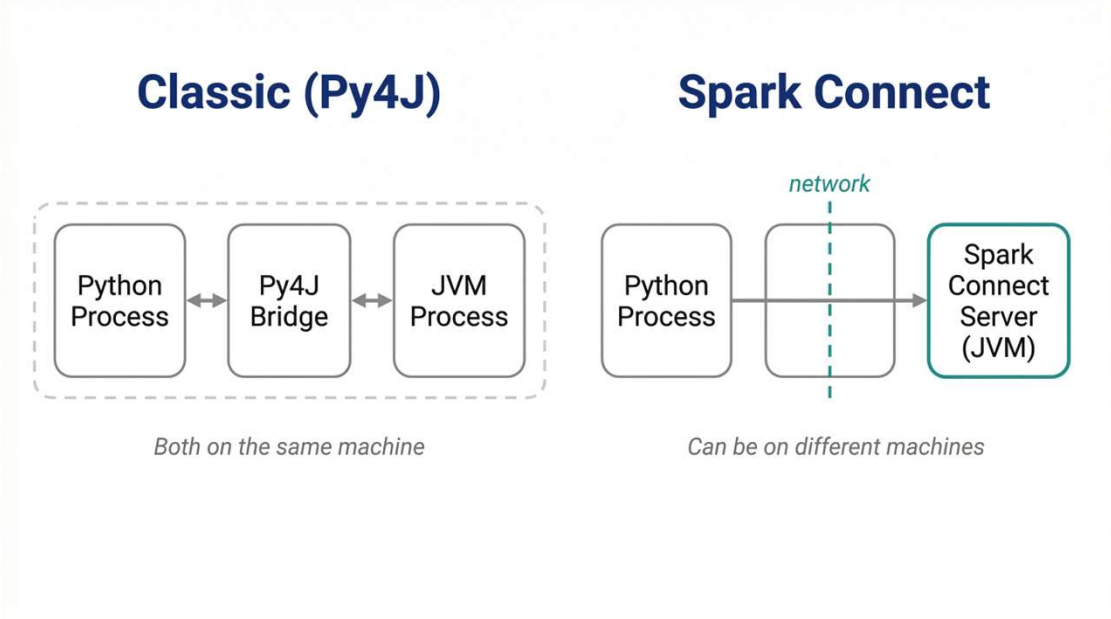
The JVM, localhost, ports, and Spark Connect

Why Does Spark Need Java?



- Spark is written in Scala → compiles to Java bytecode → runs on the JVM
- Java's WORA promise (1990s): compile once, run on any platform with a JVM
- Spark 4.0 requires JDK 17+ (the JVM is an active dependency)

Where Does Python Fit?

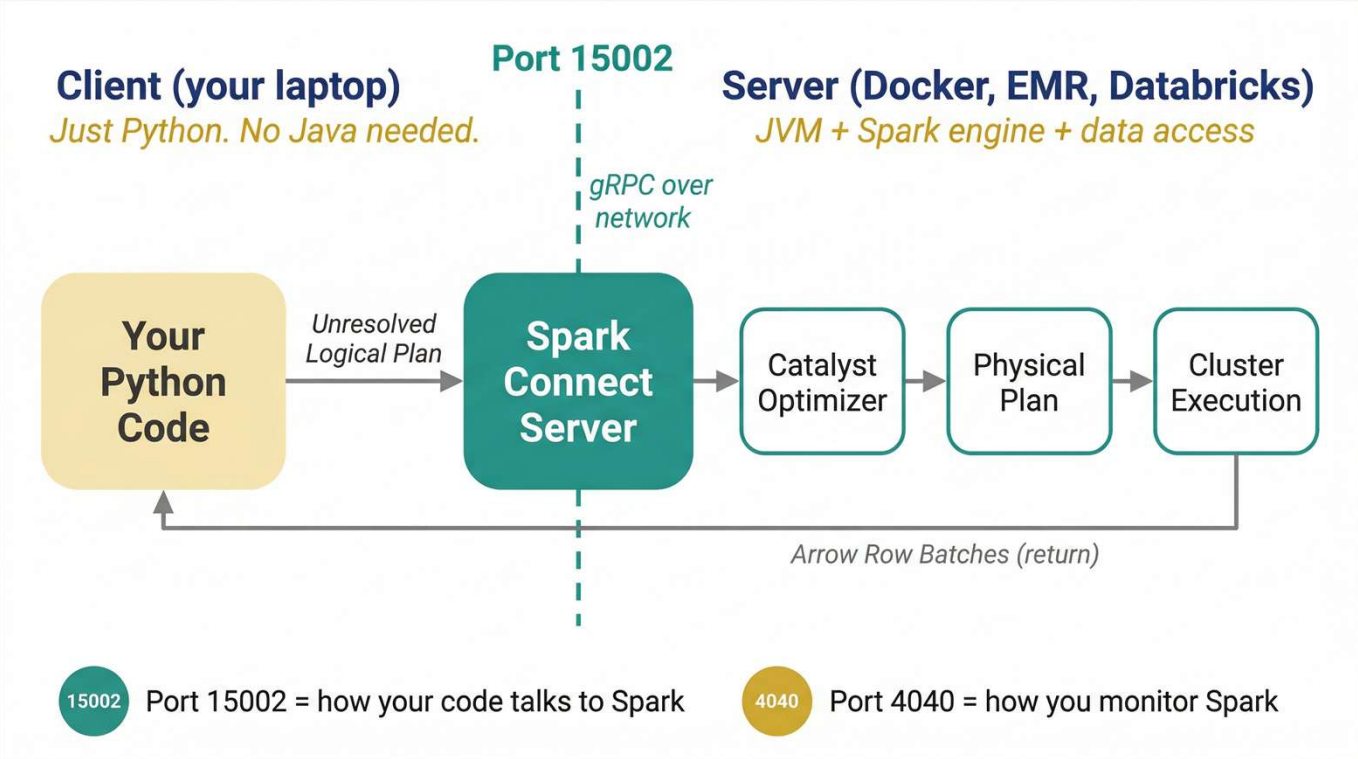


Classic (Py4J)
Python + JVM coupled on the same machine

Spark Connect
Python sends plans over gRPC to a remote Spark server. Can be different machines.

> *Python is the control plane for plans, not the data plane for per-row compute.*

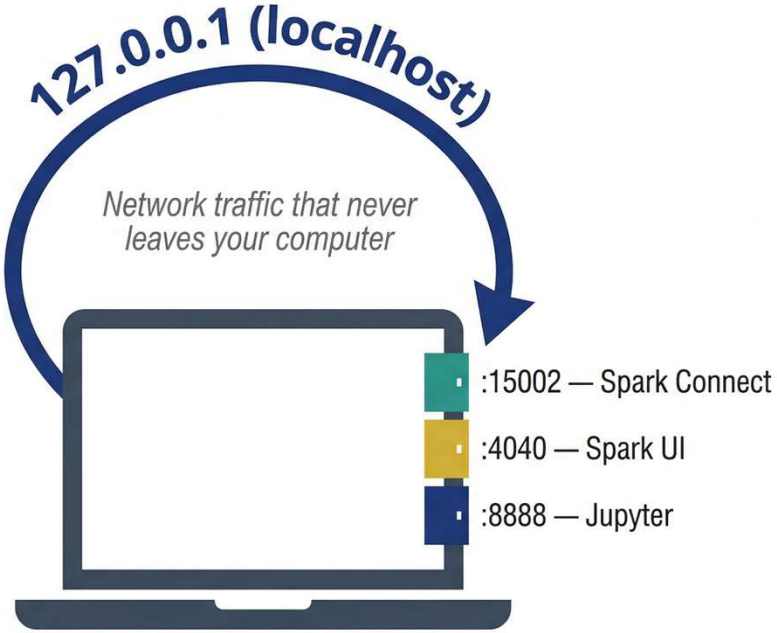
Spark Connect Architecture



1. Port **15002** = how your code talks to Spark
2. Port **4040** = how you monitor Spark

Client is just Python. Server has the JVM + engine.

Localhost and the Loopback



- 127.0.0.1 is the loopback address — traffic loops back without touching the network
- local[*] = use all CPU cores as executors. 8-core laptop = 8-executor cluster.
- Spark doesn't care if the "network" is 1,000 machines or one laptop. Same protocols, same ports.

Port Reference Card — Save This

Service	Port	What It Does
Spark Connect (gRPC)	15002	Client API — how your code talks to Spark
Spark UI (driver)	4040	Dashboard — DAGs, stages, tasks, metrics
Spark Master	7077	Cluster manager communication
Spark Master Web UI	8080	Cluster health and worker monitoring
Spark History Server	18080	Post-mortem analysis of completed apps
Jupyter Notebook	8888	Interactive notebook server
PostgreSQL	5432	Database connections (Week 7 RDS)
MySQL	3306	Database connections
MLflow Tracking UI	5000	Experiment tracking dashboard
Airflow Webserver	8080	Workflow orchestration dashboard

If you get "Address already in use," two services want the same port. Note: Spark Master and Airflow both default to 8080.

Three Ways to Run Spark in 2026

Tier 1: Traditional

- Install: JDK 17+, Spark 3.5.6, winutils (Windows)
- Set: JAVA_HOME, SPARK_HOME, PATH
- Pros: Full offline access, maximum visibility
- Cons: High setup variance, OS-specific issues

Tier 2: Docker + Connect

- Install: Docker Desktop, pip install pyspark[connect]
- Connect: sc://localhost:15002
- Pros: Reproducible, just Python on client
- Cons: Requires Docker Desktop

RECOMMENDED

Tier 3: Managed Platform

- Install: Nothing — browser-based
- Platforms: Databricks, EMR, SageMaker
- Pros: Zero setup, auto-scaling, governance
- Cons: Requires internet, less visibility, cost

Same Spark code works at all three tiers. Only the connection changes.

Docker + Spark Connect: The Fastest Path

Server (Docker)

```
docker pull apache/spark:4.0.2-scala2.13-
  java17-python3-ubuntu

docker run --rm -it \
  -p 15002:15002 \
  -p 4040:4040 \
  apache/spark:4.0.2-... \
  /opt/spark/sbin/start-connect-server.sh
```

-p 15002 maps container port to laptop

Client (Your Python)

```
pip install "pyspark[connect]==4.0.2"

from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .remote("sc://localhost:15002") \
    .getOrCreate()

df = spark.createDataFrame(
    [(1, "Alice"), (2, "Bob")],
    ["id", "name"])
df.show()
```

sc:// = Spark Connect URL scheme

Apple Silicon: Official images include ARM64 variants. No Rosetta emulation needed.

What Docker Is Actually Doing

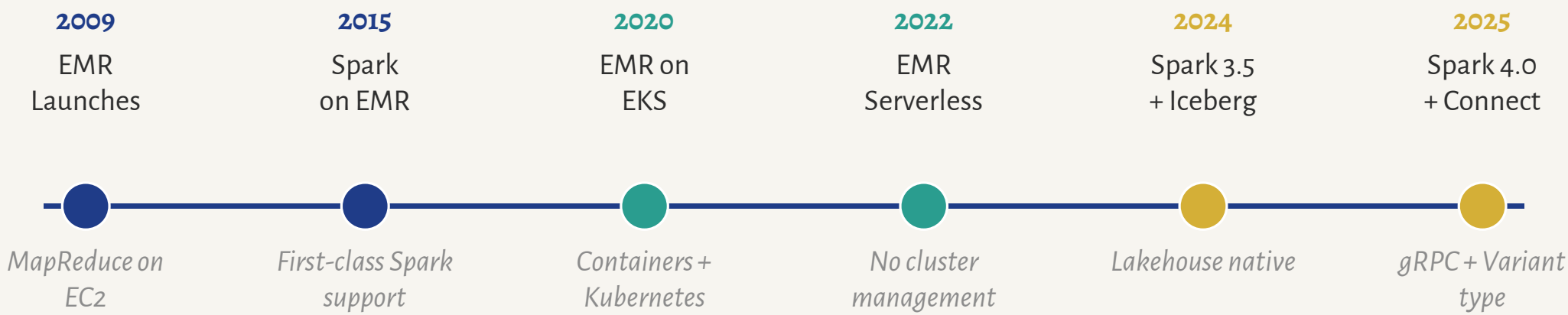
Without Docker	With Docker
Install JDK 17 manually	JDK 17 baked into image
Download Spark, set env vars	Spark pre-configured in image
Fix PATH issues per OS	Identical across Mac/Win/Linux
Version conflicts possible	Pinned versions in image tag
Port conflicts with local services	Isolated network namespace

Docker Desktop is free for education. If your employer needs it for commercial use, check the license terms.

Part 2: Amazon EMR

From Docker to AWS: Managed Hadoop and Spark

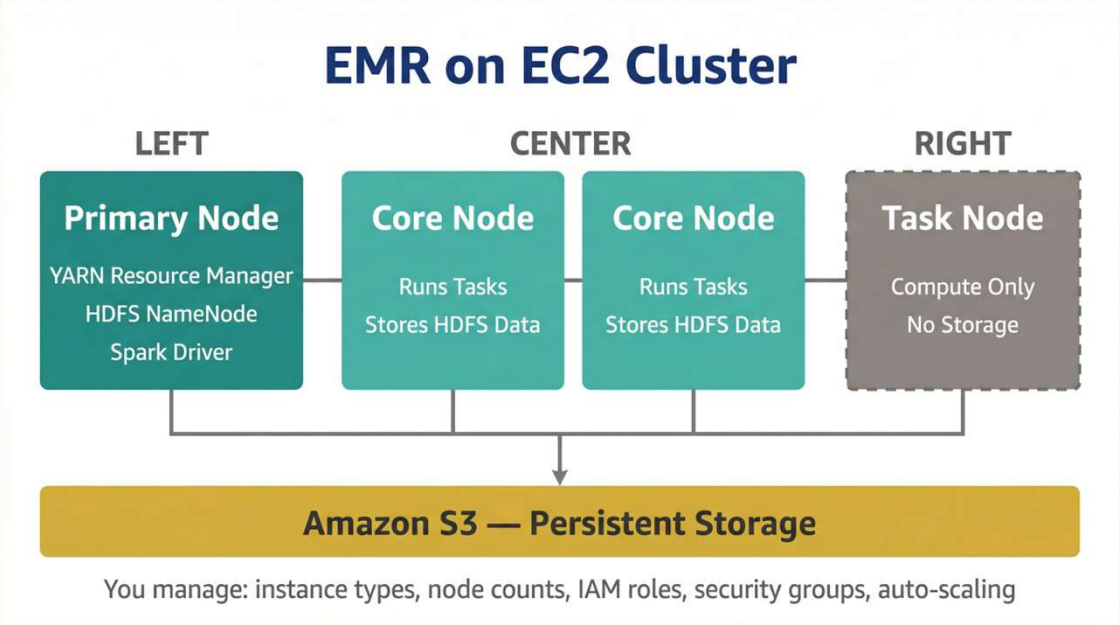
EMR Timeline: 15 Years of Managed Hadoop



Key Takeaway

The trend is clear: less infrastructure to manage, more focus on code. EMR Serverless eliminates cluster sizing. Spark Connect eliminates client-side JVM.

EMR on EC2



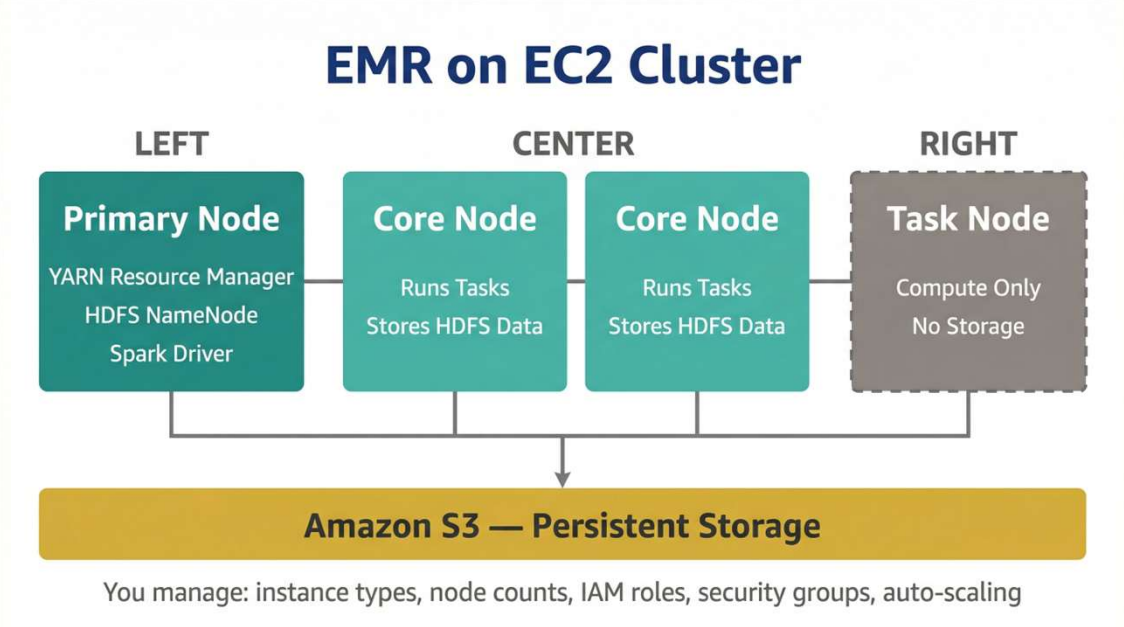
Three Node Types

- **Master** — YARN Resource Manager, HDFS NameNode
- **Core** — run tasks + store HDFS data
- **Task** — compute only, no storage

You Manage

Cluster sizing, instance types, auto-scaling policies, IAM, VPC, security groups, bootstrap actions

EMR Serverless



How It Works

- Submit a Spark job (JAR or Python script)
- AWS provisions workers automatically
- Pay only for compute time used
- Workers scale to zero when idle

You Manage

Your Spark code and IAM role. That's it.

Comparing EMR Options

Dimension	EMR on EC2	EMR Serverless
Setup	Choose instance types, cluster size	Create application, submit jobs
Scaling	Manual or auto-scaling policies	Automatic, per-job
Cost Model	Per-instance-hour (even idle)	Per-vCPU-second (only when running)
Control	SSH to nodes, custom AMIs, bootstrap	Limited: no node access
Storage	HDFS + S3	S3 only
Spark UI	Port 4040 via SSH tunnel	Pre-signed URL in console
Best For	Long-running, complex, streaming	Batch ETL, intermittent workloads

AWS Glue: ETL Without Infrastructure

Glue Crawler

Scans S3, databases, or streams and infers schema. Populates the Glue Data Catalog automatically.

Glue ETL Jobs

Serverless Spark jobs. Write PySpark or use visual editor. Auto-generates boilerplate code.

Data Catalog

Central metadata store. Tables, partitions, schemas. Shared across Athena, Redshift, EMR.

Glue vs. EMR: Glue is simpler (fully managed ETL) but less flexible. EMR gives you full Hadoop ecosystem access. Many orgs use both.

Amazon SageMaker: ML Platform Overview

Build, Train, Deploy — The Full ML Lifecycle in One Service

Build

- SageMaker Studio notebooks
- Data Wrangler for prep
- Feature Store for reusable features
- Built-in algorithms (XGBoost, etc.)

Train

- Managed training jobs
- Distributed training (multi-GPU)
- Hyperparameter tuning (Bayesian)
- Experiments for tracking runs

Deploy

- Real-time endpoints
- Batch transform for offline
- Multi-model endpoints
- Auto-scaling and A/B testing

Preview: We'll use SageMaker Studio in the Part 2 Lab

LAB 12-A

EMR on EC2: Your First Cloud Spark Cluster

What You'll Do

- Create an EMR Serverless application (Spark 3.5 runtime)
- Upload a PySpark script to S3
- Submit a job run and monitor progress in the console
- View the Spark UI via pre-signed URL
- Verify output in S3 and tear down resources

Est. time: 30–40 minutes | Cost: ~$0.72/hour

Data Governance on AWS

Lake Formation

Centralized permissions for data lakes. Column-level security, cross-account sharing, tag-based access control.

IAM + Resource Policies

Identity-based and resource-based policies. Least-privilege principle. Service-linked roles for EMR, Glue, SageMaker.

CloudTrail + Config

Audit logging of all API calls. Config rules for compliance. Automated remediation of policy violations.

Policy → Code principle: Every IAM policy, Lake Formation grant, and Config rule can be version-controlled and audited — the same GitOps approach we used for Terraform in Weeks 9–10.

When to Use What: Platform Decision Matrix

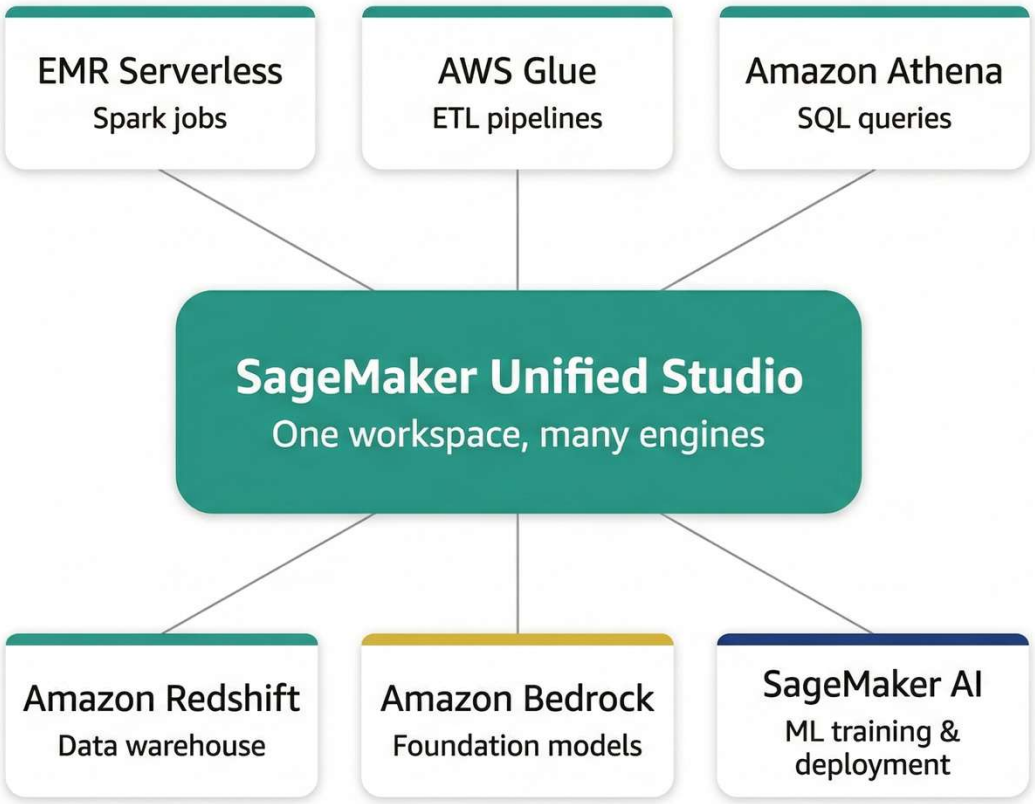
Use Case	Best Fit	Why
Simple ETL pipeline	**AWS Glue**	Fully managed, pay-per-run, visual editor
Ad hoc big data exploration	**EMR Serverless**	No cluster to manage, scales per job
Streaming / long-running	**EMR on EC2**	Persistent cluster, full Hadoop control
ML training at scale	**SageMaker**	Built-in algorithms, managed training
SQL on S3 (one-off queries)	**Athena**	Serverless SQL, no infrastructure
Local development / learning	**Docker + Connect**	Free, reproducible, instant start

PART 2

Machine Learning on AWS

SageMaker · Bedrock · Responsible AI · Cost

SageMaker Studio



JupyterLab in the Cloud

- Browser-based IDE (no local install)
- Switch compute instances without restarting
- Built-in Git, experiments, model registry
- Supports Python, R, Scala, Julia

Key Advantage

One-click access to GPU instances for training. No SSH, no Docker, no config files.

SageMaker Canvas: No-Code ML

How It Works

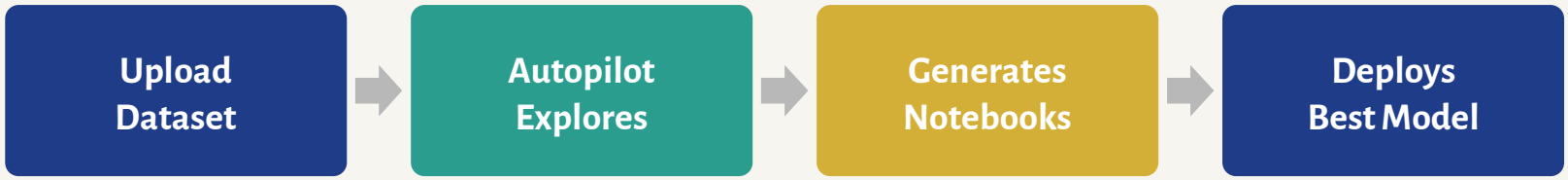
- Point-and-click: upload CSV, choose target column
- AutoML trains and compares multiple models
- One-click deploy or batch prediction
- No Python, no notebooks required

Who It's For

- Business analysts, policy researchers
- Domain experts who know data but not code
- Rapid prototyping before committing to custom ML

Think: Excel user who needs predictions

SageMaker Autopilot: AutoML



What Makes It Special

- Tries XGBoost, Linear Learner, MLP — picks the best automatically
- Generates reproducible Jupyter notebooks with full code
- Handles feature engineering, missing values, imbalanced classes
- Unlike Canvas, gives you code you can customize and version

Amazon Bedrock: Foundation Models

Text Generation

- Claude (Anthropic)
- Llama 3 (Meta)
- Command R+ (Cohere)
- Jurassic-2 (AI21)
- Titan Text (Amazon)

Image Generation

- Titan Image Generator
- Stable Diffusion (Stability AI)
- Text-to-image, image-to-image
- Watermarking built in

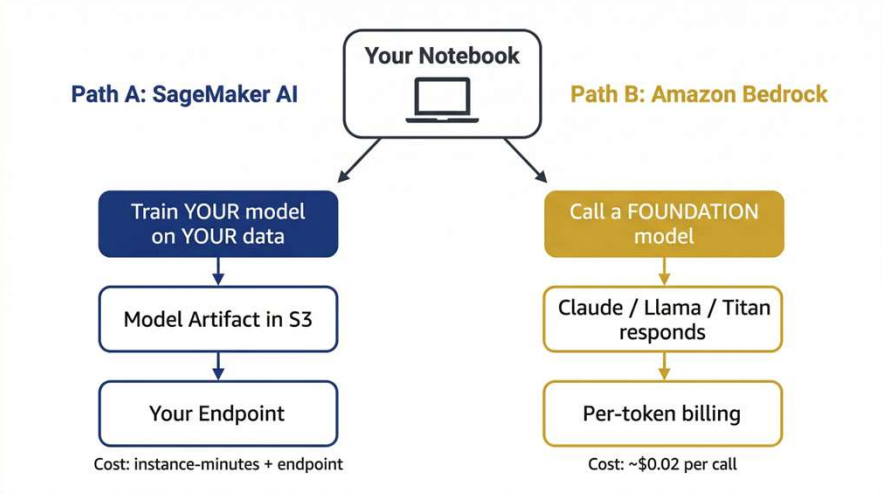
Embeddings + RAG

- Titan Embeddings
- Cohere Embed
- Knowledge Bases (auto RAG)
- S3 / OpenSearch integration

Key Difference: No training required. You call a model via API — pay per token/image. Data stays within your VPC.

Bedrock vs SageMaker: When to Choose What

	SageMaker	Bedrock
Training	Yes — full lifecycle	No — pre-trained models only
Customization	Bring your own model/data	Fine-tune or RAG
Use Case	Custom ML (tabular, vision, NLP)	Generative AI (text, images, embeddings)
Complexity	Higher — more control	Lower — API call
Cost Model	Pay for compute time	Pay per token / image
Best For	Data scientists, ML engineers	App developers, analysts



The Foundation Model Landscape

Claude 3.5
Anthropic
Best reasoning, safety, long context

Llama 3.1
Meta
Open weights, fine-tunable, 405B params

Command R+
Cohere
Grounded generation, enterprise RAG

Jurassic-2
AI21 Labs
Specialized text, custom vocabulary

Titan
Amazon
Text + image + embeddings. AWS-native.

Stable Diffusion
Stability AI
Image generation, inpainting, upscaling

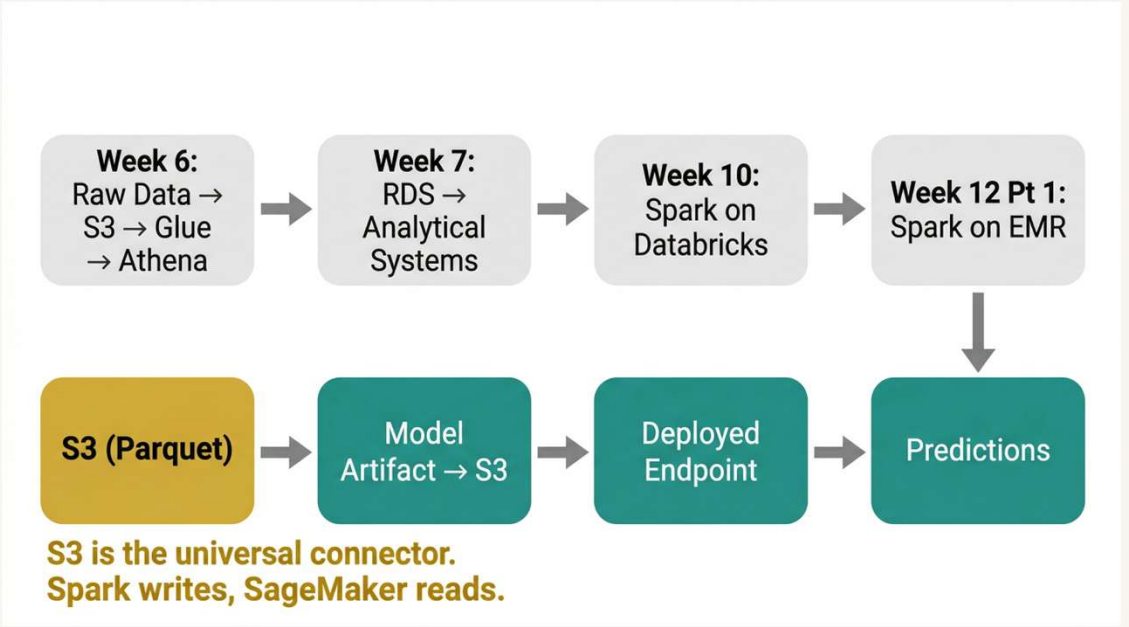
Why so many? Different models excel at different tasks. Bedrock lets you switch models without changing your code.

End-to-End ML Pipeline on AWS

Collect	Prepare	Train	Deploy	Monitor
S3, Glue Crawler	Glue ETL Athena SQL	SageMaker Autopilot	Endpoint or Batch	CloudWatch Model Monitor

What This Looks Like in Practice

- Raw CSV lands in S3 → Glue crawls schema → Athena queries for EDA
- Glue ETL cleans + writes Parquet back to S3
- Autopilot trains models, picks the best, generates notebooks
- Deploy endpoint or run batch transform; CloudWatch tracks drift



Responsible AI on AWS

SageMaker Clarify

Detect bias in training data and model predictions. Generates SHAP-based feature attribution reports.

Bedrock Guardrails

Content filters for LLM outputs. Block PII, harmful content, off-topic responses. Configurable per use case.

Model Cards

Standardized documentation for trained models. Intended use, limitations, evaluation metrics, ethical considerations.

Policy Connection: The EU AI Act and US Executive Order 14110 both require bias testing and documentation for high-risk AI systems. These tools help organizations meet those requirements programmatically.

Cost Management for ML Workloads

⚠ Common Cost Traps

- Notebook instances left running overnight
- Endpoints with no auto-scaling
- Training on GPU when CPU would suffice
- EMR clusters idle between jobs
- S3 storage class not optimized

✅ Cost Strategies

- Use Serverless when possible (EMR, Glue)
- Spot Instances for training (60–90% savings)
- Lifecycle configs to auto-stop notebooks
- Budgets + alerts in AWS Budgets
- Right-size instances (Cost Explorer)

Spot Instances and Savings Plans

Pricing Model	Savings	Risk	Best For
On-Demand	0%	None	Dev / testing / short jobs
Spot Instances	60–90%	Interruption (2 min warning)	Fault-tolerant training, EMR
Savings Plans	~40%	1–3 year commitment	Steady-state inference endpoints
Reserved	~40%	Upfront + commitment	Long-running EMR clusters

Learning AWS Without Going Broke

AWS Free Tier

- SageMaker Studio: 250 hrs/mo (2 months)
- S3: 5 GB standard storage
- Lambda: 1M requests/mo always free
- Glue: no free tier (be careful!)

AWS Academy

- $100 credit through this course
- Separate sandbox account
- Pre-configured labs (no billing surprises)
- Expires at semester end

Best Practices

- Set billing alarm at $5, $10, $25
- Check Cost Explorer weekly
- Shut down what you start
- Use Serverless by default

Key Takeaways

1. Spark separates computation from storage — Java runs the engine, Python is the interface
2. EMR Serverless removes cluster management; use it when jobs are intermittent
3. SageMaker spans the full ML lifecycle: Canvas for no-code, Studio for notebooks, Autopilot for AutoML
4. Bedrock provides managed foundation models — no training required, just API calls
5. Cost discipline is a skill: Serverless + Spot + budgets keep cloud ML affordable